

[Home](#)[GitHub](#)

CLASSES

[ErrorBoundary](#)

EVENTS

[SIGINT](#)[unhandledRejection](#)

NAMESPACES

[apiConfig](#)[contactoEmergenciaService](#)[diarioService](#)

GLOBAL

[404_Error_Handler](#)[ACTIVIDADES](#)[API_URL](#)[ASPECTOS_COGNITIVOS](#)[ActionCard](#)[ActivitySelector](#)[App](#)[AuthButtons](#)[AuthLayout](#)[Button](#)[CORS_Configuration](#)[Card](#)[Carousel](#)[Checkbox](#)[CircularProgress](#)[CognitionSelector](#)[Diario](#)[DiaryBanner](#)[DiaryEditor](#)[DiaryEntry](#)[EMOCIONES](#)[EmergencyButton](#)[EmergencyModal](#)[EmotionSelector](#)[EmotionStats](#)[Footer](#)[GET /](#)

backend/src/models/diario_mongoose.js

```
1.  /**
2.   * @file Modelo de Mongoose para las entradas del diario.
3.   * @description Define el esquema y los métodos para las entradas de
4.   * @requires mongoose
5.   * @requires bcryptjs
6.   */
7.  const mongoose = require('mongoose');
8.  const bcrypt = require('bcryptjs');
9.  const { Schema } = mongoose;
10.
11.  const SALT_WORK_FACTOR = 10;
12.
13.  /**
14.   * @typedef {object} Diario
15.   * @property {mongoose.Schema.Types.ObjectId} usuarioId - ID del usu
16.   * @property {string} titulo - Título de la entrada del diario.
17.   * @property {string} cuerpo - Contenido de la entrada del diario.
18.   * @property {string} [password] - Contraseña opcional para proteger
19.   * @property {Date} createdAt - Fecha de creación de la entrada.
20.   * @property {Date} updatedAt - Fecha de la última actualización de
21.   */
22.  const diarioSchema = new Schema({
23.    usuarioId: {
24.      type: Schema.Types.ObjectId,
25.      ref: 'User',
26.      required: true,
27.    },
28.    titulo: {
29.      type: String,
30.      required: true,
31.      trim: true,
32.    },
33.    cuerpo: {
34.      type: String,
35.      required: true,
36.    },
37.    password: {
38.      type: String,
39.      required: false, // La contraseña es opcional
40.    }
41.  }, { timestamps: true });
42.
43.  /**
44.   * @function pre-save
45.   * @description Middleware de Mongoose que hashea la contraseña ante
46.   * @description Se ejecuta solo si la contraseña ha sido modificada
47.   */
48.  // Hashear la contraseña solo si se ha proporcionado o modificado
49.  diarioSchema.pre('save', async function (next) {
50.    if (!this.isModified('password') || !this.password) {
51.      return next();
52.    }
53.  });
```

```

53.
54.     try {
55.         const salt = await bcrypt.genSalt(SALT_WORK_FACTOR);
56.         this.password = await bcrypt.hash(this.password, salt);
57.         return next();
58.     } catch (err) {
59.         return next(err);
60.     }
61. });
62.
63. /**
64.  * @method compararPassword
65.  * @description Compara una contraseña candidata con la contraseña h
66.  * @param {string} candidatePassword - La contraseña a comparar.
67.  * @returns {Promise<boolean>} - `true` si las contraseñas coinciden
68.  */
69. // Método para comparar la contraseña
70. diarioSchema.methods.compararPassword = async function (candidatePas
71.     if (!this.password) {
72.         return false; // No hay contraseña con la que comparar
73.     }
74.     return bcrypt.compare(candidatePassword, this.password);
75. };
76.
77. module.exports = mongoose.model('Diario', diarioSchema);

```

Documentation generated by [JSDoc 4.0.5](#) on Thu Dec 11 2025 11:56:27 GMT+0000
(Coordinated Universal Time) using the [docdash](#) theme.